



Dictionnaire des termes techniques



Cloud Computing & AWS

AWS

→ Plateforme cloud de Amazon fournissant stockage, calcul et réseau.

Cloud Computing

→ Utilisation de ressources informatiques via Internet.

EC2

→ Serveur virtuel dans le cloud.

EBS (Elastic Block Store)

→ Stockage bloc persistant pour serveurs cloud.

EFS (Elastic File System)

→ Système de fichiers partagé entre plusieurs serveurs.

S3 (Simple Storage Service)

→ Stockage objet pour fichiers statiques.

CloudFront

→ CDN (Content Delivery Network) pour accélérer la diffusion des contenus.



Stockage & Disques

Volume

→ Unité de stockage logique.

Snapshot

→ Copie instantanée d'un volume à un moment donné.

IOPS (Input Output Per Second)

→ Nombre d'opérations d'entrée/sortie par seconde.

Throughput

→ Débit de transfert de données.

Latency

→ Temps de réponse d'un système.

Provisioned IOPS

→ IOPS garantis configurés manuellement.

Types de Stockage EBS

gp2 / gp3

→ SSD usage général.

io1 / io2

→ SSD haute performance critique.

st1

→ HDD optimisé streaming.

sc1

→ HDD archivage froid.

Infrastructure Serveur

Instance

→ Machine virtuelle cloud.

Load Balancer

→ Répartiteur de charge réseau.

Auto Scaling

→ Ajustement automatique du nombre de serveurs.

Availability Zone (AZ)

→ Centre de données physique dans une région cloud.

Region

→ Zone géographique cloud.

Sécurité Cloud

IAM (Identity and Access Management)

→ Gestion des identités et permissions.

MFA (Multi Factor Authentication)

→ Authentification multi-facteurs.

KMS (Key Management Service)

→ Gestion des clés de chiffrement.

Encryption

→ Chiffrement des données.

Audit Log

→ Journalisation des actions système.

CloudTrail

→ Service d'audit AWS.

GuardDuty

→ Détection intelligente des menaces.

DevOps & Automatisation

Infrastructure as Code (IaC)

→ Gestion infrastructure via code.

Terraform

→ Outil d'automatisation infrastructure cloud.

CI/CD Pipeline

→ Processus automatique de développement et déploiement.

Deployment

→ Mise en production d'une application.

Automation

→ Exécution automatique des tâches.



Linux (Commandes & Système)

Filesystem

→ Structure d'organisation des fichiers.

Mount

→ Montage d'un stockage sur un dossier.

Umount

→ Démontage du stockage.

fstab

→ Fichier de montage automatique.

lsblk

→ Liste des disques disponibles.

mkfs

→ Création de système de fichiers.

resize2fs

→ Redimensionnement filesystem Linux.

chmod

→ Gestion permissions fichiers.

chown

→ Changement propriétaire fichier.

 Architecture Web**Web Server**

→ Serveur web (Apache, Nginx).

Application Server

→ Serveur applicatif.

CDN

→ Réseau de distribution de contenu.

Static Files

→ Fichiers non dynamiques (images, CSS, JS).

Uploads

→ Fichiers envoyés par utilisateurs.

 Architecture Haute Disponibilité

High Availability (HA)

→ Système conçu pour rester disponible.

Disaster Recovery (DR)

→ Plan de reprise après sinistre.

RTO (Recovery Time Objective)

→ Temps maximum de récupération.

RPO (Recovery Point Objective)

→ Perte maximale de données acceptable.



Monitoring & Performance

CloudWatch

→ Service monitoring AWS.

Metrics

→ Indicateurs de performance.

Monitoring

→ Surveillance système.

Alerting

→ Notification automatique d'erreur.



Bases de Données (Usage Cloud)

Transaction Database

→ Base de données transactionnelle.

Cluster Database

→ Ensemble de serveurs bases de données.

Replication

→ Duplication des données.



DevOps Production

Scalability

→ Capacité à évoluer avec la charge.

Cost Optimization

→ Optimisation des coûts.

Microservices

→ Architecture applicative distribuée.

Concepts Sécuritaires Avancés

Threat Model

→ Analyse des menaces.

Zero Trust

→ Modèle sécurité sans confiance implicite.

Network Isolation

→ Isolation réseau.

Résumé Global

Cloud

- | Compute
 - | Storage
 - | Network
 - | Security
 - | Automation
-

Niveau Expert (Bonus)

Les 5 concepts les plus importants en production :

- ✓ Sécurité
- ✓ Performance
- ✓ Haute disponibilité

- ✓ Automatisation
- ✓ Monitoring

Dictionnaire Ultra Expert Cloud & Infrastructure

Cloud Architecture Advanced

Cloud Native

→ Applications conçues spécialement pour le cloud.

Hybrid Cloud

→ Combinaison cloud public + infrastructure locale.

Multi Cloud

→ Utilisation de plusieurs fournisseurs cloud.

Serverless

→ Exécution code sans gestion serveur.

Containerization

→ Isolation applications via conteneurs.

Orchestration

→ Gestion automatique conteneurs (ex: Kubernetes).

Edge Computing

→ Traitement données proche utilisateur.

Fog Computing

→ Extension edge + cloud.

Architecture Logicielle

Monolith

→ Application unique massive.

Microservices

→ Services indépendants.

Event Driven Architecture

→ Architecture basée événements.

CQRS

→ Séparation lecture / écriture.

Event Sourcing

→ Stockage historique événements.

Hexagonal Architecture

→ Architecture ports/adapters.

 Storage Engineering (Ultra Expert)**Block Storage**

→ Stockage disque brut.

Object Storage

→ Stockage via objets (S3).

File Storage

→ Système fichiers partagé.

Storage Tiering

→ Hiérarchisation stockage.

Cold Storage

→ Archivage bas coût.

Hot Storage

→ Accès rapide.

Storage Replication

→ Copie données multi zones.

RAID

→ Redondance disque.

Erasure Coding

→ Protection données distribuée.

Performance Engineering

Throughput Optimization

→ Optimisation débit.

Latency Optimization

→ Réduction temps réponse.

Queue Management

→ Gestion files traitement.

Backpressure

→ Contrôle surcharge système.

Caching Layer

→ Couche cache.

Redis Cache

→ Cache mémoire rapide.

CDN Edge Cache

→ Cache réseau global.

Sécurité Ultra Avancée

Zero Trust Architecture

→ Aucun accès de confiance par défaut.

Least Privilege Principle

→ Permissions minimales.

Defense in Depth

→ Sécurité multicouche.

Security Token Service (STS)

→ Gestion accès temporaires.

HSM (Hardware Security Module)

→ Sécurité cryptographique matérielle.

Penetration Testing

→ Test intrusion.

Threat Intelligence

→ Analyse menaces globales.

SIEM

→ Analyse sécurité centralisée.

Networking Expert Level

VPC (Virtual Private Cloud)

→ Réseau isolé cloud.

Subnet

→ Sous réseau logique.

CIDR

→ Notation réseau IP.

BGP Routing

→ Routage Internet dynamique.

DNS Resolution

→ Traduction nom domaine → IP.

Load Balancing Algorithms

- Round Robin
 - Least Connection
 - Weighted Distribution
-

Haute Disponibilité

Active-Active Architecture

→ Deux systèmes actifs simultanés.

Active-Passive Architecture

→ Système backup passif.

Failover

→ Bascule automatique.

Chaos Engineering

→ Test résistance système.



DevOps & Automation Expert

GitOps

→ Gestion infra via Git.

Infrastructure Pipeline

→ Automatisation déploiement.

Immutable Infrastructure

→ Infrastructure non modifiée après déploiement.

Blue Green Deployment

→ Deux environnements production.

Canary Deployment

→ Déploiement progressif.



Container & Kubernetes Expert

Docker

→ Conteneurisation applications.

Kubernetes

→ Orchestration conteneurs.

Pod

→ Unité déploiement Kubernetes.

Service Mesh

→ Gestion communication microservices.

Istio

→ Service mesh populaire.

Helm

→ Gestion packages Kubernetes.



Database Expert

ACID Properties

- Atomicity
- Consistency
- Isolation
- Durability

CAP Theorem

- Consistency
- Availability
- Partition Tolerance

Sharding

→ Partition base données.

Replication Lag

→ Retard synchronisation.

Read Replica

→ Base lecture uniquement.



Monitoring & Observability

Observability

→ Comprendre état système via logs.

Prometheus

→ Monitoring métrique.

Grafana

→ Visualisation données.

Tracing Distributed

→ Suivi requêtes distribuées.

OpenTelemetry

→ Standard observabilité.

FinOps (Optimisation coûts cloud)

Cost Governance

→ Gestion budgets cloud.

Right Sizing

→ Dimensionnement ressources.

Reserved Instances

→ Réduction coût long terme.

Spot Instances

→ Instances économiques.

Compliance & Audit

GDPR

→ Protection données Europe.

ISO 27001

→ Standard sécurité.

PCI DSS

→ Sécurité paiement.

Data Sovereignty

→ Localisation données.

Advanced Storage Performance (EBS Expert)

IO Queue Depth

→ File d'attente opérations disque.

Block Size Optimization

→ Taille bloc stockage.

Striping

→ Distribution données.

Multi Attach Volume

→ Partage volume multi instances.



AI / Future Cloud Technologies

MLOps

→ Machine learning production.

Data Lake

→ Stockage données brutes.

Data Warehouse

→ Analyse structurée.

Stream Processing

→ Traitement temps réel.



Top 20 Concepts Architecte Senior

1. Zero Trust
2. Multi Region DR
3. Microservices
4. Observability
5. IaC
6. Security Automation
7. CI/CD
8. Scalability Design
9. Distributed Systems

- 10. Event Streaming
 - 11. Chaos Testing
 - 12. Encryption Everywhere
 - 13. Immutable Deployments
 - 14. Cost Optimization
 - 15. Container Orchestration
 - 16. Service Mesh
 - 17. Edge Computing
 - 18. AI Operations (AIOps)
 - 19. Data Governance
 - 20. Compliance Engineering
-

Conclusion

Un architecte cloud expert doit maîtriser 5 piliers :

Architecture

Security

Performance

Automation

Cost Optimization

Type de projet	AWS Service	Rôle / Usage	Avantages	Limites / Points à savoir	Coût approximatif / Free Tier
Portfolio statique / site vitrine	S3 + CloudFront	Hébergement site HTML/CSS/JS	Simple, scalable, CDN intégré, sécurisé	Pas de backend dynamique sans Lambda	Gratuit jusqu'à 5 Go stockage + 50 Go transfert / mois
	AWS Amplify	Hosting + CI/CD pour projets React/Vue/Angular	Déploiement rapide, intégré à GitHub, backend serverless possible	Limité aux projets front-end / besoin de config backend séparée	Gratuit : 5 Go stockage, 15 Go bande passante / mois
Blog / petit CMS	Lightsail (WordPress)	Serveur préconfiguré avec WordPress	Facile à configurer, tout inclus (PHP + MySQL)	Moins flexible qu'EC2 pour gros projets	5\$/mois pour le plan de base
Petite boutique / e-commerce	Lightsail + MySQL	Hébergement WooCommerce / Laravel	Simple, stable, tout inclus	Trafic élevé peut nécessiter upgrade	5–10\$/mois selon plan
	Amplify + AppSync + Lambda	Frontend React/Vue + backend serverless	Serverless, scalable, paiement à l'usage	Courbe d'apprentissage plus haute	Gratuit si faible trafic

Backend léger / API	Lambda + API Gateway	Fonctions serverless pour API ou formulaires	Aucun serveur à gérer, très économique	Pas adapté aux traitements lourds en continu	Gratuit : 1 million de requêtes / mois
Base de données NoSQL	DynamoDB	Stockage des données (produits, utilisateurs)	Serverless, scalable, performant	Pas relationnel, coûts peuvent croître si gros trafic	Gratuit : 25 Go, 200M req/mois
Base de données SQL	RDS (MySQL / PostgreSQL)	Base relationnelle pour backend plus robuste	Gestion automatisée, backups	Plus cher que Lightsail, Free Tier limité	Gratuit 750h / mois + 20 Go stockage pour 12 mois
Authentification / utilisateurs	Cognito	Gestion login / signup, JWT, OAuth	Simple, sécurisé, intégré aux apps mobile/web	Interface moins flexible que solutions custom	Gratuit jusqu'à 50 000 utilisateurs actifs / mois
Stockage fichiers / images	S3	Stockage d'images ou documents	Très scalable, intégration facile avec Lambda	Nécessite configuration permissions	Gratuit : 5 Go / mois
Monitoring / Logs	CloudWatch	Suivi des fonctions Lambda, API, serveur	Alertes et métriques faciles	Peut devenir coûteux si beaucoup de logs	Gratuit : 10 alarmes, 5 Go logs / mois
Nom de domaine	Route 53	Gestion DNS et domaines	Intégré AWS, fiable	Tarif à part (~0,50\$/mois par	Payant

				domaine + requêtes DNS)	
Certificat SSL / HTTPS	AWS Certificat e Manager	Certificats TLS gratuits	Sécurise site gratuitem ent, intégration n CloudFro nt	Fonctionne uniquement avec certains services AWS	Gratuit






EC2

Tableau de bord

Vue globale EC2 ↗

Événements

▼ Instances

Instances

Types d'instances

Paires de clés

Lancer une instance

Pour commencer, lancez une instance Amazon EC2 qui est un serveur virtuel dans le cloud.

Lancer une instance ▼

Migrer un serveur ↗

▼ Récapitulatif

Nombre d'instances

[Informations](#)

1

Image logicielle (AMI)

Amazon Linux 2023 AMI 2023.10....[en savoir plus](#)
ami-0c1fe732b5494dc14

Type de serveur virtuel (type d'instance)

t3.micro

Pare-feu (groupe de sécurité)

[Annulez](#)

Lancer l'instance

Créer une paire de clés ou continuer sans paire de clés



 Nous avons remarqué que vous n'avez pas sélectionné de paire de clés. Si vous souhaitez pouvoir vous connecter à votre instance, il est recommandé d'en créer une ou de sélectionner une paire de clé déjà existante.

☒ Créer une paire de clés

☐ Continuer sans paire de clés

Nom de la paire de clés

Les paires de clés vous permettent de vous connecter à votre instance en toute sécurité.

ibrahimadiagneheck

La longueur maximale du nom est de 255 caractères ASCII. Il ne peut pas inclure d'espaces avant ou après.

[Annulez](#)

[Lancer l'instance](#)

ibrahimadiagneheck.pem

▼ Nom et balises Informations

Clé Informations

Q Saisir la clé

Valeur Informations

Q Web01 X

Types de ressources Informations

Sélectionner les types de ress... ▼

Supprimer

Instances X

Volumes X

Interfaces réseau X

☐ Masquer tous les éléments sélectionnés

Clé Informations

Q Project X

Valeur Informations

Q Titan X

Types de ressources Informations

Sélectionner les types de ress... ▼

Supprimer

Instances X

Volumes X

Interfaces réseau X

☐ Masquer tous les éléments sélectionnés

Clé Informations

Q Env X

Valeur Informations

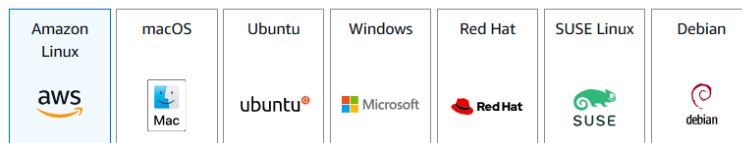
Q Dev X

Types de ressources Informations

Sélectionner les types de ress... ▼

Supprimer

Instances X



Explorer plus d'AMI
Y compris les AMI d'AWS, de Marketplace et de la communauté

Amazon Machine Image (AMI)

AMI Amazon Linux 2023 kernel-6.1
ami-0c1fe732b5494dc14 (64 bits (x86), uefi-preferred) / ami-02b86da1e539b4dd0 (64 bits (Arm), uefi)
Virtualisation: hvm ENA activé: true Type de périphérique racine: ebs

Éligible à l'offre gratuite ▼

Description

Amazon Linux 2023 (kernel-6.1) est un système d'exploitation moderne à usage général basé sur Linux qui bénéficie de 5 ans de support à long terme. Il est optimisé pour AWS et conçu pour fournir un environnement d'exécution sécurisé, stable et haute performance permettant de développer et d'exécuter vos applications cloud.

Amazon Linux 2023 AMI 2023.10.20260202.2 x86_64 HVM kernel-6.1

Architecture

64 bits (x86) ▼

Mode de démarrage

uefi-preferred

ID AMI

ami-0c1fe732b5494dc14

Date de publication

2026-02-03

Nom d'utilisateur ⓘ

ec2-user

Fournisseur vérifié

aws Recherche [Alt+S] États-Unis (Virginie du

EC2 > Instances > i-06bef1a1970a03c94 > Connectez-vous à l'instance

ID d'instance i-06bef1a1970a03c94 ID de VPC vpc-0a24393e3c69cdfc2 Groupes de sécurité sg-0631d05aa42014140 (launch-wizard-2) Rôle IAM -

EC2 Instance Connect SSM Session Manager Client SSH EC2 Serial Console

ID d'instance i-06bef1a1970a03c94

- Ouvrez un client SSH.
- Recherchez votre fichier de clé privée. La clé utilisée pour lancer cette instance est `ibrahimadiagneeck.pem`
- Exécutez, si nécessaire, cette commande pour vous assurer que votre clé n'est pas visible publiquement.
`chmod 400 "ibrahimadiagneeck.pem"`
- Connectez-vous à votre instance à l'aide de son DNS public :
`ec2-34-229-134-149.compute-1.amazonaws.com`

Exemple :
`ssh -i "ibrahimadiagneeck.pem" ec2-user@ec2-34-229-134-149.compute-1.amazonaws.com`

Remarque : Dans la plupart des cas, le nom d'utilisateur deviné est correct. Cependant, lisez les instructions d'utilisation de l'AMI pour vérifier si le propriétaire de l'AMI a modifié le nom d'utilisateur par défaut.

aws Recherche [Alt+S] Posez une question à Amazon Q

EC2 > Instances > i-06bef1a1970a03c94 > Connectez-vous à l'instance

ID d'instance i-06bef1a1970a03c94 ID de VPC vpc-0a24393e3c69cdfc2 Groupes de sécurité sg-0631d05aa42014140 (launch-wizard-2) Rôle IAM -

EC2 Instance Connect SSM Session Manager Client SSH EC2 Serial Console

ec2-user@ip-172-31-28-199:~\$ ssh -i "ibrahimadiagneeck.pem" ec2-user@ec2-34-229-134-149.compute-1.amazonaws.com

ibrah@ibrah MINGW64 ~/Documents/GitHub/DevOps-certification/aws/vm (main)
\$ chmod 400 "ibrahimadiagneeck.pem"

ibrah@ibrah MINGW64 ~/Documents/GitHub/DevOps-certification/aws/vm (main)
\$ ssh -i "ibrahimadiagneeck.pem" ec2-user@ec2-34-229-134-149.compute-1.amazonaws.com

The authenticity of host 'ec2-34-229-134-149.compute-1.amazonaws.com (34.229.134.149)' can't be established.
ED25519 key fingerprint is SHA256:5BH8D1TeCGAKuVMABox5VvdQ6WNi00akKVkaNs1wAuw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

ssh -i "ibrahimadiagneeck.pem" ec2-user@34.229.134.149

Lancer une instance | EC2 | us-east-1 console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-0631d05aa42014140 - Scoaire

(67) Apprendre à lire... SpeakyReads - Impr... Votre travail - Bitbu... Courrier - idseck@d... hkhcoder/vprofile-p... Annuler - Angular 1... Cours : Devenir Dév... Tous les favoris

aws Recherche [Alt+S] Posez une question à Amazon Q États-Unis (Virginie du Nord) Ibrahim (6815-2627-7265) ibrahimadiagneeck

EC2 > Groupes de sécurité > sg-0631d05aa42014140 - launch-wizard-2 > Modifier les règles entrantes

Modifier les règles entrantes

Les règles entrantes contrôlent le trafic entrant qui est autorisé à atteindre l'instance.

Règles entrantes	Type	Protocole	Plage de ports	Source	Description - facultatif
sg-011ebcbd5ec7d734f	SSH	TCP	22	Personn...	
-	TCP personnalisé	TCP	80	N'impor...	
-	TCP personnalisé	TCP	80	N'impor...	

Ajouter une règle

Les règles dont la source est 0.0.0.0/0 ou ::/0 permettent à toutes les adresses IP d'accéder à votre instance. Nous vous recommandons de paramétrer les règles du groupe de sécurité de sorte que les accès soient uniquement autorisés depuis des adresses IP connues.

Annuler Aperçu des modifications Enregistrer les règles

CloudShell Commentaires Console de l'application mobile © 2026, Amazon Web Services, Inc. ou ses affiliés. Confidentialité Conditions Préférences relatives aux cookies 21:27 11/02/2026

- Rendre statique le "public ip" : voir menu elastic ips
- Règles entrant sélectionnées et accorder tout ceux sortant
- Bloquer les suppression d'instance
-

RAPPORT — Normes professionnelles de création d'instances AWS (EC2)

1. Introduction

Dans les environnements cloud modernes, notamment sur AWS, la création d'instances de calcul (EC2) doit respecter des normes strictes afin de garantir :

La sécurité des systèmes

L'optimisation des coûts

La traçabilité des ressources

La maintenabilité des infrastructures

L'utilisation des TAGS, des règles de sécurité réseau, et d'une architecture standardisée est aujourd'hui une exigence professionnelle.

2. Objectifs des normes AWS

Objectif	Description
----------	-------------

Sécurité	Limiter accès non autorisés
----------	-----------------------------

Organisation	Classifier les ressources
--------------	---------------------------

Performance	Adapter les ressources au besoin
-------------	----------------------------------

Coût	Éviter le gaspillage
------	----------------------

Audit	Suivi et conformité
-------	---------------------

3. Norme de nommage (Naming Convention)

 Format recommandé

<type>-<service>-<environment>

Exemples :

Nom	Signification
-----	---------------

web-nginx-prod	serveur web production
----------------	------------------------

api-spring-dev	backend test
----------------	--------------

db-mysql-prod	base données
---------------	--------------

4. TAGS AWS (obligatoires en entreprise)

 Structure standard

Tag	Rôle	Exemple
-----	------	---------

Name	Nom lisible	web-prod-server
------	-------------	-----------------

Project	Projet	gestion-vehicules
---------	--------	-------------------

Owner	Responsable	Idrissa
-------	-------------	---------

Environment	Environnement	dev
-------------	---------------	-----

CostCenter	Budget	IT-2025
------------	--------	---------

CreatedDate	Date	2026-02-15
-------------	------	------------

 Exemple complet réel

Name: api-spring-prod

Project: gestion-vehicules-douanes

Owner: Idrissa

Environment: prod

CostCenter: DSI-Cloud

CreatedDate: 2026-02-15

5. Normes de sécurité (CRITIQUE)

🔒 5.1 Accès serveur

Élément	Norme
---------	-------

Authentification	Clé SSH uniquement
------------------	--------------------

Root	Désactivé
------	-----------

MFA AWS	Activé
---------	--------

🔥 5.2 Security Group standard

Port	Accès	Pourquoi
------	-------	----------

22	IP admin uniquement	SSH
----	---------------------	-----

80	Public	HTTP
----	--------	------

443	Public	HTTPS
-----	--------	-------

3306	Privé seulement	Base données
------	-----------------	--------------

6. Norme stockage

Paramètre	Standard
-----------	----------

Type	gp3
------	-----

Taille	20–50 Go
--------	----------

Snapshot	Hebdomadaire
----------	--------------

Chiffrement	Activé
-------------	--------

7. Normes de coûts

Bonnes pratiques :

✓ Instances adaptées (t3.micro pour petits projets)

✓ Arrêt auto hors horaires

✓ Monitoring usage

8. Exemple d'architecture professionnelle simple

Utilisateur



Load Balancer



EC2 Nginx



EC2 Backend Java



RDS MySQL

9. Checklist création instance AWS

✓ Choisir type adapté

✓ Ajouter tags obligatoires

✓ Sécuriser ports

- ✓ Clé SSH
- ✓ Chiffrement disque
- ✓ Snapshot
- ✓ Monitoring

10. Conclusion

Le respect des normes AWS permet :

Une infrastructure sécurisée

Des coûts maîtrisés

Une organisation professionnelle

Une évolutivité facile

Ces standards sont indispensables dans les environnements critiques comme les systèmes administratifs (ex : douanes).



1. Exemple complet de TAGS AWS (standard pro)

Clé (Key)	Valeur (Value)
Name	web-nginx-prod
Project	boutique-angul ar
Owner	Idrissa
Environme nt	prod
CostCenter	IT-CLOUD-202 6
CreatedDat e	2026-02-15
Application	ecommerce

Backup	weekly
Criticality	high

✓ Version brute (comme dans AWS)

Name=web-nginx-prod
Project=boutique-angular
Owner=Idrissa
Environment=prod
CostCenter=IT-CLOUD-2026
CreatedDate=2026-02-15
Application=ecommerce
Backup=weekly
Criticality=high



2. Exemple données instance EC2

Paramètre	Valeur
Instance Type	t3.micro
OS	Ubuntu 22.04
Region	eu-west-3
Storage	30 Go gp3
Public IP	Oui
Monitoring	Activé
Auto shutdown	22h



3. Exemple Security Group propre

Port	Source	Usage
------	--------	-------

22	197.xxx.xxx.10	SSH admin
80	0.0.0.0/0	Web public
443	0.0.0.0/0	HTTPS
3306	10.0.0.0/16	Base privée

4. Exemple architecture projet réel

Projet : boutique-angular

Serveur	Rôle	Environnement
web-nginx-prod	Front Angular	prod
api-spring-prod	Backend Java	prod
db-mysql-prod	Base données	prod

5. Exemple suivi coûts (mensuel)

Ressource	Projet	Coût approx
web-nginx-prod	boutique-angular	8\$
api-spring-prod	boutique-angular	12\$
db-mysql-prod	boutique-angular	18\$

6. Exemple naming convention appliqué

type-service-environment

Exemples :

web-nginx-prod

api-spring-dev

db-postgres-prod

batch-worker-test

Si tu veux, je peux te fournir :

- ✓ Données pour 10–20 serveurs (comme vraie entreprise)
- ✓ Modèle Excel de gestion cloud
- ✓ Script AWS CLI avec tags automatiques
- ✓ Exemple audit sécurité

- Avec IAM donner un utilisateur des accès spécifiques à des services spécifiques.



```
PS C:\Users\ibrah> aws --version
aws-cli/2.33.22 Python/3.13.11 Windows/11 exe/AMD64
PS C:\Users\ibrah> aws configure
AWS Access Key ID [None]: AKIAZ5LRBWSI3KC7SEYC
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: json
PS C:\Users\ibrah> |
```

```
Default output format [None]: json
PS C:\Users\ibrah> ls C:\Users\ibrah\.aws\config
```

Répertoire : C:\Users\ibrah\.aws

Mode	LastWriteTime	Length	Name
-a----	16/02/2026 22:43	46	config

```
PS C:\Users\ibrah> ls ~\.aws\config
```

Répertoire : C:\Users\ibrah\.aws

Mode	LastWriteTime	Length	Name
-a----	16/02/2026 22:43	46	config

```
PS C:\Users\ibrah> ls ~\.aws\
```

Répertoire : C:\Users\ibrah\.aws

Mode		LastWriteTime	Length	Name
----		-----	-----	----
-a----	16/02/2026	22:43	46	config
-a----	16/02/2026	22:43	119	credentials

```
PS C:\Users\ibrah> aws sts get-caller-identity
{
  "UserId": "AIDAZ5LRBWSIZDJEM4JMV",
```

```
PS C:\Users\ibrah> aws ec2 describe-instances
{
  "Reservations": []
}
```

RAPPORT COMPLET – Gestion du stockage EBS

1 Introduction

Le service **Amazon Elastic Block Store (EBS)** est un service de stockage bloc fourni par **Amazon Web Services** et utilisé principalement avec les instances **Amazon EC2**.

EBS permet de :

- Stocker des données applicatives
- Héberger bases de données
- Gérer fichiers système
- Stocker images et fichiers web

Caractéristiques :

- Persistant
 - Haute performance
 - Redimensionnable
 - Snapshot possible
-

2 Types de volumes EBS

◆ **SSD (Performance élevée)**

★ **gp3 (Recommandé en production)**

- Usage général

- Coût optimisé
- IOPS configurable

★ gp2

- Ancienne génération
- Performance dépend de la taille

★ io1

- Applications critiques
- Bases de données transactionnelles

★ io2

- Ultra haute disponibilité
- Production critique

◆ HDD (Stockage massif)

st1

- Big data
- Streaming

sc1

- Archivage froid
-

3 Architecture du stockage

Structure logique :

Instance EC2

↓

EBS Volume

↓

Filesystem (ext4, xfs)

↓

Dossier application

4 Commandes AWS CLI – Gestion EBS

Lister volumes

```
aws ec2 describe-volumes
```

Filtrer par instance :

```
aws ec2 describe-volumes \
--filters Name=attachment.instance-id,Values=i-xxxxx
```

Créer volume

```
aws ec2 create-volume \
--size 20 \
--availability-zone us-east-1a \
--volume-type gp3
```

Attacher volume

```
aws ec2 attach-volume \  
--volume-id vol-xxxxx \  
--instance-id i-xxxxx \  
--device /dev/sdf
```



Détacher volume

```
aws ec2 detach-volume \  
--volume-id vol-xxxxx
```



Modifier volume

Augmenter taille :

```
aws ec2 modify-volume \  
--volume-id vol-xxxxx \  
--size 50
```

Changer type :

```
aws ec2 modify-volume \  
--volume-id vol-xxxxx \  
--volume-type gp3
```



Supprimer volume

```
aws ec2 delete-volume \  
--volume-id vol-xxxxx
```

5 Snapshots (Sauvegarde)

Créer snapshot

```
aws ec2 create-snapshot \  
--volume-id vol-xxxxx \  
--description "backup"
```

Restaurer volume depuis snapshot

```
aws ec2 create-volume \  
--snapshot-id snap-xxxxx \  
--availability-zone us-east-1a
```

6 Gestion côté Linux après attachement

Vérifier disque

```
lsblk
```

Formater volume

```
sudo mkfs -t ext4 /dev/xvdf
```

Monter volume

```
sudo mkdir /data  
sudo mount /dev/xvdf /data
```

Monter automatiquement

Dans `/etc/fstab` :

```
/dev/xvdf /data ext4 defaults,nofail 0 2
```

Tester :

```
sudo mount -a
```

Étendre filesystem

Après extension EBS :

```
sudo resize2fs /dev/xvdf
```

7 Attachement sur volume existant

Vérification état volume

```
aws ec2 describe-volumes --volume-ids vol-xxxxx
```

State doit être :

```
available
```

Attachement

```
aws ec2 attach-volume \  
--volume-id vol-xxxxx \  
--instance-id i-xxxxx \  
--device /dev/sdf
```

8 Attachement pour site web (Images)

Cas pratique : site web avec dossier uploads.

Objectif :

```
/var/www/html/uploads
```

Étapes

1. Attacher volume

Comme précédemment.

2. Formater si nouveau volume

```
sudo mkfs.ext4 /dev/xvdf
```

3. Monter sur dossier web

```
sudo mkdir -p /var/www/html/uploads  
sudo mount /dev/xvdf /var/www/html/uploads
```

4. Permissions serveur web

Apache :

```
sudo chown -R www-data:www-data /var/www/html/uploads  
sudo chmod -R 755 /var/www/html/uploads
```

9 Cas architecture web moderne

Pour un site scalable :

Load Balancer

↓

EC2 Instances



EFS ou S3

Stockage partagé recommandé :

- **Amazon EFS** → stockage partagé
- **Amazon S3** → images et fichiers statiques

10 Comparaison EBS vs autres stockages

Service	Usage
EBS	Disque serveur
EFS	Partage multi-serveurs
S3	Stockage objet web

11 Bonnes pratiques production

- ✓ Toujours snapshot avant modification
- ✓ Utiliser gp3 par défaut
- ✓ Séparer OS et données
- ✓ Surveiller avec CloudWatch
- ✓ Utiliser EFS pour architecture web distribuée

12 Scénarios réels

✓ Base de données

- io2 recommandé

✓ Site web

- EBS pour serveur unique
- EFS pour cluster

✓ Backup

- Snapshot automatisé
-

13 Sécurité

- ✓ Chiffrer volumes
 - ✓ Contrôler IAM permissions
 - ✓ Limiter accès SSH
 - ✓ Automatiser backup
-

Conclusion

Le service EBS est essentiel pour le stockage bloc sur AWS.
La maîtrise des commandes CLI permet :

- Gestion complète du cycle de vie stockage
- Haute disponibilité
- Scalabilité
- Sécurité des données

★ Niveau Expert (Conseil Production)

Architecture recommandée :

S3 → fichiers statiques

EFS → fichiers partagés

EBS → base de données

Snapshots → backup

RAPPORT EXPERT – Storage Cloud Production

1 Architecture avancée du stockage cloud

Dans **Amazon Elastic Compute Cloud**, le stockage est généralement structuré ainsi :

Application Layer



Compute Layer (EC2)



Storage Layer (EBS / EFS / S3)

Solutions complémentaires :

- **Amazon Simple Storage Service** → fichiers statiques
- **Amazon Elastic File System** → partage multi-serveurs
- **EBS** → stockage bloc haute performance

2 Types de volumes – Choix production

★ General Purpose SSD

👉 gp3 = recommandé par défaut
Usage :

- Applications web
- Serveurs applicatifs

★ Provisioned IOPS SSD

👉 io1 / io2

Usage :

- Bases transactionnelles
- ERP
- Banque

Avantages :

- Latence faible
 - Performance garantie
-

★ HDD Optimized

👉 st1 / sc1

Usage :

- Big data
 - Backup
 - Archivage
-

3 Performance Tuning (Expert)

🔥 Paramètres critiques

IOPS (Input Output Per Second)

Détermine :

- vitesse lecture/écriture

Commande monitoring :

```
aws cloudwatch get-metric-statistics
```

Surveiller :

- latency
- throughput
- queue depth

Bonnes configurations

Usage	Type recommandé
Web server	gp3
Database	io2
Backup	sc1
Streaming	st1

4 Sécurité (Niveau production critique)

Chiffrement

Activer encryption :

```
aws ec2 create-volume \  
--encrypted \  
--kms-key-id alias/aws/ebs
```

Utilise le service KMS de AWS.

Contrôle IAM

Restreindre accès :

- Création volume
- Suppression volume
- Modification

Politique minimale recommandée.

Protection données

Toujours appliquer :

- Snapshot automatique
 - Versioning backup
 - Monitoring anomalies
-

5 Automatisation DevOps

Terraform (Infrastructure as Code)

Exemple :

```
resource "aws_ebs_volume" "data" {  
  availability_zone = "us-east-1a"  
  size             = 50  
  type             = "gp3"  
}
```

Avantages :

- Reproductibilité
 - Versioning infrastructure
-

CI/CD Integration

Pipeline typique :

Code push

↓

Build

↓

Test

↓

Deploy

↓

Attach storage

6 Architecture Web Haute Disponibilité

Pour applications distribuées :

Load Balancer

↓

Auto Scaling Group

↓

EC2 Instances

↓

EFS (shared storage)

Pourquoi ?

👉 EBS = attaché à une seule instance

👉 EFS = partagé entre serveurs

7 Gestion images site web (Production réelle)

Cas e-commerce :

Utilisateur

↓

Upload image

↓

Application serveur

↓

Stockage S3

Pourquoi S3 ?

- Haute disponibilité
- CDN compatible
- Coût faible

Exemple upload :

```
aws s3 cp image.jpg s3://bucket-name/images/
```

8 Backup et Disaster Recovery

Snapshot automatique

```
aws ec2 create-snapshot \  
--volume-id vol-xxxxx
```

Stratégie recommandée :

Type	Fréquence
Production	1h
Database	15 min
Archive	24h

9 Monitoring Production

Utiliser :

- CloudWatch metrics
- Alarmes automatiques
- Logs centralisés

Alertes recommandées :

- CPU > 80%
 - Latency élevée
 - Disk full
-

10 Haute Disponibilité & Scalabilité

Architecture entreprise :

Region 1

└ Availability Zone A

└ Availability Zone B

Region 2 (DR Backup)

Stratégie DR (Disaster Recovery) :

- Cross-region snapshot replication

Coût Optimisation (Important en entreprise)

Conseils :

- ✓ Utiliser gp3
- ✓ Supprimer snapshots inutiles
- ✓ Automatiser lifecycle
- ✓ Dimensionner correctement IOPS

Outils :

- AWS Cost Explorer

Problèmes fréquents en production

Problème	Solution
Volume plein	Extend filesystem
Latency	Changer type volume
Perte données	Snapshot
Crash instance	Auto scaling

13 Commandes Expert utiles

```
lsblk  
df -h  
iotop  
iostat
```

Surveillance performance disque.

14 Cas réels entreprise

✓ E-commerce

- S3 → images
- EBS → bases données

✓ Banque

- io2 → transactions

✓ SaaS

- EFS → stockage partagé
-

Conclusion Expert

Le stockage sur AWS repose sur une bonne séparation :

Compute → EC2

Block Storage → EBS

File Storage → EFS

Object Storage → S3

La réussite d'une architecture dépend de :

- Performance
- Sécurité
- Coût
- Haute disponibilité

Recommandation Architecte Cloud

Stack moderne :

CloudFront CDN

↓

S3 Static Content

↓

EC2 + EBS Database

↓

EFS Shared Storage

↓

Snapshots Backup

RAPPORT EXPERT SENIOR – EBS Sécurité & Production Critique

1 Modèle de sécurité avancé

Sécurité basée sur 4 couches :

IAM Security

↓

Network Security (VPC)

↓

Storage Encryption

↓

Monitoring & Audit

Services associés :

- IAM policies
- KMS encryption
- CloudTrail audit logs

2 Chiffrement avancé des données

Utiliser **KMS (Key Management Service)**.

Chiffrement au niveau :

- Volume
- Snapshot
- Transit

Création volume chiffré

```
aws ec2 create-volume \
```

```
--size 100 \  
--availability-zone us-east-1a \  
--encrypted \  
--kms-key-id alias/aws/ebs
```

Protection clé

Utiliser :

- Rotation clé automatique
- Key policy restrictive

Exemple politique :

```
Allow only EC2 roles to decrypt
```

3 Attaques possibles et protections

✗ Attaque 1 : Accès non autorisé IAM

Protection :

- MFA obligatoire
 - Rôles temporaires
-

✗ Attaque 2 : Exfiltration données

Protection :

- Encryption obligatoire
- Network isolation

❌ Attaque 3 : Snapshot theft

Protection :

```
aws ec2 modify-snapshot-attribute
```

Limiter partage snapshot.

4 Performance Ultra Tuning (Niveau Expert)

🔥 Paramètres critiques

Paramètre	Impact
IOPS	Performance transaction
Throughput	Débit données
Queue depth	Latence

Monitoring Performance

Utiliser :

- CloudWatch Metrics
- Application monitoring

Commandes :

```
aws cloudwatch list-metrics
```

Optimisation Base de données

Pour bases critiques :

- io2 Block Express
 - Multi-attach volumes
-

5 Architecture Banque / Gouvernement

Architecture recommandée :

Internet



Firewall



Load Balancer



App Servers (EC2)



EBS io2



Database Cluster

Haute sécurité :

Ajouter :

- WAF
 - Bastion Host
 - Private Subnets
-

6 Disaster Recovery (DRP réel entreprise)

Stratégies DR

Strategy	RTO	RPO
Backup Snapshot	Moyen	Moyen
Cross Region Replication	Excellent	Faible
Multi Region	Meilleur	Minimal

Automatisation Snapshot

Script cron :

```
aws ec2 create-snapshot \  
--volume-id vol-xxxxx \  
--description "auto backup"
```

Planification :

- Production → 1h
 - Banque → 15 minutes
-

7 Haute Disponibilité (HA)

Architecture entreprise moderne :

```
Region A  
├ AZ1  
└ AZ2
```


└ AZ3

Region B (DR Backup)

Services utilisés :

- Auto Scaling
- Elastic Load Balancer

8 DevOps Automation (Production réelle)

Terraform – Infrastructure complète

```
resource "aws_ebs_volume" "prod" {  
  availability_zone = "us-east-1a"  
  size             = 200  
  type             = "io2"  
  iops             = 3000  
}
```

CI/CD Pipeline

Pipeline sécurisé :

Git

↓

Code Review

↓

Security Scan

↓

Build

↓

Deploy



Attach Storage

9 Surveillance avancée (SOC Security)

Outils recommandés :

- CloudTrail
- GuardDuty
- Security Hub

Audit logs :

`aws cloudtrail lookup-events`

10 Optimisation Coût (Entreprise)

Stratégies :

- ✓ Lifecycle snapshots
 - ✓ Auto deletion snapshots anciens
 - ✓ Dimensionnement intelligent
-

Exemple policy lifecycle

- Snapshot < 30 jours → garder

- 30 jours → archive

11 Comparaison stockage entreprise

Service	Usage
EBS	Serveur unique
EFS	Partage multi serveurs
S3	Object storage

12 Cas Production Réel (Entreprise SaaS)

Stack moderne :

CloudFront CDN

↓

S3 Static Files

↓

EC2 Application Layer

↓

EBS Database Storage

↓

RDS Database Service

13 Monitoring avancé performance disque

Commandes Linux :

`iostat -x`

`iotop`

`vmstat`

Mesures :

- Wait time
- Read latency
- Write latency

Erreurs critiques production

Erreur	Solution
Disk full	Extend volume
Instance failure	DR Recovery
Snapshot failure	Check IAM

Architecture Cloud 2026 recommandée (Ultra moderne)

User



CDN (CloudFront)



Microservices Containers



Kubernetes Cluster



EBS Persistent Storage



Backup Snapshot + S3 Archive

Conclusion Niveau Expert

EBS est un composant critique des infrastructures cloud modernes.

Architecture idéale :

Security Layer



Network Isolation



Storage Encryption



Backup Strategy



Monitoring + Automation

La réussite dépend de :

- Sécurité
- Performance
- Automatisation
- Haute disponibilité

Rapport : Comprendre les concepts de Serveur, Proxy, Endpoint et Load Balancer

1 Le Serveur

♦ Définition

Un **serveur** est une machine (physique ou virtuelle) qui fournit des services, des données ou des ressources à d'autres machines appelées **clients**.

♦ Types de serveurs

- Serveur Web (ex: Apache HTTP Server, Nginx)
- Serveur d'application (ex: Apache Tomcat)
- Serveur de base de données (ex: MySQL, PostgreSQL)
- Serveur de fichiers
- Serveur mail

♦ Rôle

- Stocker les données
 - Exécuter des applications
 - Répondre aux requêtes des clients (via HTTP, TCP, etc.)
-

2 Le Proxy

♦ Définition

Un **proxy** est un intermédiaire entre le client et le serveur.

👉 Il reçoit la requête du client et la transmet au serveur, puis retourne la réponse.

♦ **Types de proxy**

♦ **Proxy direct (Forward Proxy)**

- Protège les clients
- Cache les requêtes
- Contrôle l'accès Internet

♦ **Reverse Proxy**

- Protège les serveurs
- Gère la sécurité
- Peut faire du load balancing

Exemple de reverse proxy :

- Nginx
- HAProxy

♦ **Fonctions principales**

- Sécurité (masquage IP)
- Cache
- Compression
- SSL/TLS termination

3 Le Load Balancer

♦ **Définition**

Un **load balancer** (répartiteur de charge) est un système qui distribue le trafic réseau entre plusieurs serveurs.

♦ Pourquoi l'utiliser ?

- Améliorer la performance
- Éviter la surcharge d'un serveur
- Assurer la haute disponibilité
- Tolérance aux pannes

♦ Types

Type	Description
Layer 4	Basé sur IP et ports
Layer 7	Basé sur HTTP, URL, cookies

♦ Exemples

- HAProxy
- Nginx
- AWS Elastic Load Balancing

4 L'Endpoint

♦ Définition

Un **endpoint** est un point d'accès à un service réseau.

Dans une API, un endpoint correspond à une URL spécifique.

♦ Exemple

<https://api.monsite.com/users>

Ici :

- `api.monsite.com` → serveur
- `/users` → endpoint

♦ Types d'endpoints

- REST API
- SOAP API
- Microservices

Un endpoint est donc :

- 📌 L'adresse précise où un service est accessible

Architecture complète : Comment tout fonctionne ensemble ?

Schéma simplifié

`Client` → `Proxy / Load Balancer` → `Serveurs` → `Base de données`

Étapes :

1. Le client envoie une requête
2. Le proxy reçoit la requête
3. Le load balancer choisit un serveur disponible
4. Le serveur traite la requête

5. Réponse envoyée au client

Sécurité et Haute Disponibilité

Élément	Rôle principal
Serveur	Exécute l'application
Proxy	Sécurité et filtrage
Load Balancer	Répartition du trafic
Endpoint	Point d'accès au service

Comparaison simple

Concept	Image simple
Serveur	Un restaurant
Endpoint	Un plat spécifique du menu
Proxy	Le réceptionniste
Load Balancer	Le responsable qui répartit les clients entre plusieurs restaurants

Conclusion

Ces éléments sont essentiels dans toute architecture moderne :

-  Applications Web
-  Cloud Computing

-  Systèmes d'entreprise
-  Applications mobiles

Ils permettent :

- Performance
- Sécurité
- Scalabilité
- Haute disponibilité

Rapport enrichi : Infrastructure Réseau Moderne et Sécurité

Rappel : Le modèle OSI

Le modèle OSI (Open Systems Interconnection) comporte 7 couches :

Couche	Nom	Rôle
7	Application	HTTP, FTP, API
6	Présentation	Chiffrement SSL/TLS
5	Session	Gestion des sessions
4	Transport	TCP / UDP
3	Réseau	IP
2	Liaison de données	MAC
1	Physique	Câbles, signaux

👉 Chaque composant réseau agit à une ou plusieurs couches.

Network Load Balancer (NLB)

♦ Définition

Un **Network Load Balancer (NLB)** est un répartiteur de charge fonctionnant principalement à la **couche 4 (Transport)** du modèle OSI.

Exemple Cloud :

- AWS Network Load Balancer

♦ **Caractéristiques**

- Très haute performance
- Ultra faible latence
- Support TCP / UDP
- Gestion de millions de requêtes par seconde
- IP statique possible

♦ **Couche OSI concernée**

Élément	Couche
Network Load Balancer	Couche 4 (Transport)

Il prend ses décisions selon :

- Adresse IP
- Port
- Protocole

Il ne lit **pas** le contenu HTTP.

♦ **Cas d'usage**

- Applications temps réel

- Jeux en ligne
 - Streaming
 - Systèmes bancaires
 - Microservices haute performance
-

3 Gateway Load Balancer (GWLB)

♦ Définition

Un **Gateway Load Balancer (GWLB)** permet d'insérer des **appliances de sécurité** dans le flux réseau.

Exemple :

- AWS Gateway Load Balancer
-

♦ Rôle principal

Il redirige le trafic vers :

- Pare-feu
 - IDS
 - IPS
 - Appliances de sécurité
-

♦ Couche OSI

Principalement :

Fonction	Couche
Routage IP	Couche 3
Inspection transport	Couche 4

♦ Pourquoi il est important ?

Il permet :

- Inspection transparente
 - Scalabilité des équipements de sécurité
 - Séparation réseau / sécurité
 - Haute disponibilité
-

4 Détection d'intrusion (IDS / IPS)

IDS – Intrusion Detection System

- Surveille le trafic
- Détecte les comportements suspects
- Génère des alertes

Exemple :

- Snort
-

IPS – Intrusion Prevention System

- Bloque automatiquement les attaques
- Analyse en temps réel
- Peut couper la connexion

◆ Couche OSI impliquée

Type d'analyse	Couche
IP	Couche 3
TCP/UDP	Couche 4
Inspection HTTP	Couche 7

Les IDS/IPS modernes font de la **Deep Packet Inspection (DPI)** → inspection profonde des paquets.

5 Intégration complète dans une architecture

Architecture enrichie

Client



Network Load Balancer (L4)



Gateway Load Balancer



IDS / IPS / Firewall



Reverse Proxy (L7)



Serveurs applicatifs



Base de données

6 Répartition des rôles par couche OSI

Composant	Couche OSI	Fonction
Network Load Balancer	4	Répartition TCP/UDP
Gateway Load Balancer	3-4	Distribution vers appliances
IDS / IPS	3-7	Inspection et sécurité
Reverse Proxy	7	Gestion HTTP
Serveur Web	7	Traitement applicatif

7 Différence claire entre NLB et GWLB

Critère	NLB	GWLB
But	Répartition trafic	Intégration sécurité
Couche	4	3-4
Analyse contenu	✗ Non	✓ Oui (via appliances)
Cas d'usage	Performance	Sécurité avancée

8 Notion stratégique : Sécurité en couches (Defense in Depth)

L'intégration :

- NLB
- GWLB
- IDS/IPS
- Proxy
- Firewall

Permet d'appliquer une **défense en profondeur** :

1. Répartition intelligente
 2. Inspection réseau
 3. Inspection applicative
 4. Contrôle d'accès
 5. Chiffrement TLS
-

9 Impact en Cloud et DevOps

Dans le Cloud (ex: Amazon Web Services) :

- Scalabilité automatique
- Sécurité managée
- Monitoring centralisé

- Haute disponibilité multi-AZ
-

10 Conclusion technique

L'architecture moderne repose sur :

-  Couche 4 → Performance (NLB)
-  Couche 3-7 → Sécurité (GWLB + IDS/IPS)
-  Couche 7 → Logique applicative (Proxy / Serveur)

L'objectif est :

- Haute disponibilité
- Sécurité renforcée
- Scalabilité horizontale
- Résilience aux attaques

